** Check models **

In [7]:
```
import requests
import os

api_key = “I deleted my API key. Place your owned API key here”

headers = {
    "Authorization": f"Bearer {api_key}",
    "Content-Type": "application/json"
}

response = requests.get(url, headers=headers)

print(response.json())
```

{'object': 'list', 'data': [{'id': 'openai/gpt-oss-safeguard-20b', 'object': 'model', 'created': 1761708789, 'owned_by': 'OpenAI', 'active': True, 'context_window': 131072, 'public_apps': None, 'max_completion_tokens': 65536, 'hugging_face_id': 'openai/gpt-oss-safeguard-20b', 'name': 'Safety GPT OSS 20B', 'input_modalities': ['text'], 'output_modalities': ['text'], 'context_length': 131072, 'max_output_length': 65536, 'pricing': {'prompt': '0.000000075', 'completion': '0.0000003', 'image': '0', 'request': '0', 'input_cache_read': '0.0000000375'}, 'supported_sampling_parameters': ['temperature', 'top_p', 'stop', 'seed', 'max_tokens'], 'supported_features': ['tools', 'json_mode', 'structured_outputs', 'reasoning']}, {'id': 'allam-2-7b', 'object': 'model', 'created': 1737672203, 'owned_by': 'SDAIA', 'active': True, 'context_window': 4096, 'public_apps': None, 'max_completion_tokens': 4096, 'hugging_face_id': 'ALLaM-AI/ALLaM-2.0-7B-Instruct', 'name': 'ALLaM-2-7b', 'input_modalities': ['text'], 'output_modalities': ['text'], 'context_length': 4096, 'max_output_length': 4096, 'supported_sampling_parameters': ['temperature', 'top_p', 'stop', 'seed', 'max_tokens'], 'supported_features': ['json_mode']}, {'id': 'meta-llama/llama-prompt-guard-2-22m', 'object': 'model', 'created': 1748632101, 'owned_by': 'Meta', 'active': True, 'context_window': 512, 'public_apps': None, 'max_completion_tokens': 512, 'name': 'Llama Prompt Guard 2 22M', 'input_modalities': ['text'], 'output_modalities': ['text'], 'context_length': 512, 'max_output_length': 512, 'pricing': {'prompt': '0.00000003', 'completion': '0.00000003', 'image': '0', 'request': '0', 'input_cache_read': '0.000000015'}, 'supported_sampling_parameters': ['temperature', 'top_p', 'stop', 'seed', 'max_tokens']}, {'id': 'llama-3.3-70b-versatile', 'object': 'model', 'created': 1733447754, 'owned_by': 'Meta', 'active': True, 'context_window': 131072, 'public_apps': None, 'max_completion_tokens': 32768, 'hugging_face_id': 'meta-llama/Llama-3.3-70B-Instruct', 'name': 'Llama 3.3 70B', 'input_modalities': ['text'], 'output_modalities': ['text'], 'context_length': 131072, 'max_output_length': 32768, 'pricing': {'prompt': '0.00000059', 'completion': '0.00000079', 'image': '0', 'request': '0', 'input_cache_read': '0.000000295'}, 'supported_sampling_parameters': ['temperature', 'top_p', 'stop', 'seed', 'max_tokens'], 'supported_features': ['tools', 'json_mode']}, {'id': 'openai/gpt-oss-20b', 'object': 'model', 'created': 1754407957, 'owned_by': 'OpenAI', 'active': True, 'context_window': 131072, 'public_apps': None, 'max_completion_tokens': 65536, 'hugging_face_id': 'openai/gpt-oss-20b', 'name': 'GPT OSS 20B', 'input_modalities': ['text'], 'output_modalities': ['text'], 'context_length': 131072, 'max_output_length': 65536, 'pricing': {'prompt': '0.000000075', 'completion': '0.0000003', 'image': '0', 'request': '0', 'input_cache_read': '0.0000000375'}, 'supported_sampling_parameters': ['temperature', 'top_p', 'stop', 'seed', 'max_tokens'], 'supported_features': ['tools', 'json_mode', 'structured_outputs', 'reasoning']}, {'id': 'whisper-large-v3-turbo', 'object': 'model', 'created': 1728413088, 'owned_by': 'OpenAI', 'active': True, 'context_window': 448, 'public_apps': None, 'max_completion_tokens':

448, 'hugging_face_id': 'openai/whisper-large-v3-turbo', 'name': 'Whisper Large V3 Turbo', 'input_modalities': ['audio'], 'output_modalities': ['transcription'], 'context_length': 448, 'max_output_length': 448}, {'id': 'whisper-large-v3', 'object': 'model', 'created': 1693721698, 'owned_by': 'OpenAI', 'active': True, 'context_window': 448, 'public_apps': None, 'max_completion_tokens': 448, 'hugging_face_id': 'openai/whisper-large-v3', 'name': 'Whisper', 'input_modalities': ['audio'], 'output_modalities': ['transcription'], 'context_length': 448, 'max_output_length': 448}, {'id': 'canopylabs/orpheus-arabic-saudi', 'object': 'model', 'created': 1765926439, 'owned_by': 'Canopy Labs', 'active': True, 'context_window': 4000, 'public_apps': None, 'max_completion_tokens': 50000, 'name': 'Canopy Labs Orpheus Arabic Saudi', 'input_modalities': ['text'], 'output_modalities': ['speech'], 'context_length': 4000, 'max_output_length': 50000, 'pricing': {'prompt': '0.00004', 'image': '0', 'request': '0', 'input_cache_read': '0.00002'}}, {'id': 'groq/compound-mini', 'object': 'model', 'created': 1756949707, 'owned_by': 'Groq', 'active': True, 'context_window': 131072, 'public_apps': None, 'max_completion_tokens': 8192, 'hugging_face_id': 'groq/compound-beta-mini', 'name': 'Compound Mini', 'input_modalities': ['text'], 'output_modalities': ['text'], 'context_length': 131072, 'max_output_length': 8192, 'supported_sampling_parameters': ['temperature', 'top_p', 'stop', 'seed', 'max_tokens'], 'supported_features': ['json_mode']}, {'id': 'canopylabs/orpheus-v1-english', 'object': 'model', 'created': 1766186316, 'owned_by': 'Canopy Labs', 'active': True, 'context_window': 4000, 'public_apps': None, 'max_completion_tokens': 50000, 'name': 'Canopy Labs Orpheus V1 English', 'input_modalities': ['text'], 'output_modalities': ['speech'], 'context_length': 4000, 'max_output_length': 50000, 'pricing': {'prompt': '0.000022', 'image': '0', 'request': '0', 'input_cache_read': '0.000011'}}, {'id': 'llama-3.1-8b-instant', 'object': 'model', 'created': 1693721698, 'owned_by': 'Meta', 'active': True, 'context_window': 131072, 'public_apps': None, 'max_completion_tokens': 131072, 'hugging_face_id': 'meta-llama/Llama-3.1-8B-Instruct', 'name': 'Llama 3.1 8B', 'input_modalities': ['text'], 'output_modalities': ['text'], 'context_length': 131072, 'max_output_length': 131072, 'pricing': {'prompt': '0.00000005', 'completion': '0.00000008', 'image': '0', 'request': '0', 'input_cache_read': '0.000000025'}, 'supported_sampling_parameters': ['temperature', 'top_p', 'stop', 'seed', 'max_tokens'], 'supported_features': ['tools', 'json_mode']}, {'id': 'openai/gpt-oss-120b', 'object': 'model', 'created': 1754408224, 'owned_by': 'OpenAI', 'active': True, 'context_window': 131072, 'public_apps': None, 'max_completion_tokens': 65536, 'hugging_face_id': 'openai/gpt-oss-120b', 'name': 'GPT OSS 120B', 'input_modalities': ['text'], 'output_modalities': ['text'], 'context_length': 131072, 'max_output_length': 65536, 'pricing': {'prompt': '0.00000015', 'completion': '0.0000006', 'image': '0', 'request': '0', 'input_cache_read': '0.000000075'}, 'supported_sampling_parameters': ['temperature', 'top_p', 'stop', 'seed', 'max_tokens'], 'supported_features': ['tools', 'json_mode', 'structured_outputs', 'reasoning']}, {'id': 'meta-llama/llama-prompt-guard-2-86m', 'object': 'model', 'created': 1748632165, 'owned_by': 'Meta', 'active': True, 'context_window': 512, 'public_apps': None, 'max_completion_tokens': 512, 'name': 'Prompt Guard 2 86M', 'input_modalities': ['text'], 'output_modalities': ['text'], 'context_length': 512, 'max_output_length': 512, 'pricing': {'prompt': '0.00000004', 'completion': '0.00000004', 'image': '0', 'request': '0', 'input_cache_read': '0.00000002'}, 'supported_sampling_parameters': ['temperature', 'top_p', 'stop', 'seed', 'max_tokens'], 'supported_features': ['json_mode']}, {'id': 'groq/compound', 'object': 'model', 'created': 1756949530, 'owned_by': 'Groq', 'active': True, 'context_window': 131072, 'public_apps': None, 'max_completion_tokens': 8192, 'hugging_face_id': 'groq/compound-beta', 'name': 'Compound', 'input_modalities': ['text'], 'output_modalities': ['text'], 'context_length': 131072, 'max_output_length': 8192, 'supported_sampling_parameters': ['temperature', 'top_p', 'stop', 'seed', 'max_tokens'], 'supported_features': ['json_mode']}, {'id': 'qwen/qwen3.6-27b', 'object': 'model', 'created':

1778288776, 'owned_by': 'Alibaba Cloud', 'active': True, 'context_window': 131072, 'public_apps': None, 'max_completion_tokens': 16384, 'hugging_face_id': 'Qwen/Qwen3.6-27B', 'name': 'Qwen/Qwen3.6-27B', 'input_modalities': ['text', 'image'], 'output_modalities': ['text'], 'context_length': 131072, 'max_output_length': 16384, 'pricing': {'prompt': '0.0000006', 'completion': '0.000003', 'image': '0', 'request': '0', 'input_cache_read': '0.0000003'}, 'supported_sampling_parameters': ['temperature', 'top_p', 'stop', 'seed', 'max_tokens'], 'supported_features': ['tools', 'json_mode', 'reasoning']}]}

** Canvas 1 **

In [1]:

```
# ======================================================
# IMPORTS
# ======================================================

import json
import asyncio
import os
import pandas as pd

from groq import AsyncGroq

from bert_score import score
from rouge_score import rouge_scorer


# ======================================================
# CONFIG
# ======================================================

# Khuyến nghị:
# export GROQ_API_KEY="gsk_xxxxxxxxxxxxxxxxxxxxxxxx"

GROQ_API_KEY = “I deleted my API key. Place your owned API key here”

# Hoặc để thử nhanh:
# GROQ_API_KEY = "gsk_xxxxxxxxxxxxxxxxxxxxxxxx"

if not GROQ_API_KEY:
    raise RuntimeError(
        "Environment variable GROQ_API_KEY not found."
    )


# ======================================================
# MODEL
# ======================================================

MODEL_NAME = "qwen/qwen3.6-27b"


# ======================================================
# FILE PATHS
# ======================================================

TEST_FILE = "./Training_Data/test.json"

CHECKPOINT_CSV = (
    "./Training_Data/"
    "groq_qwen36_checkpoint.csv"
```

```
)

OUTPUT_CSV = (
    "./Training_Data/"
    "Baseline_Groq_Qwen3.6-27B_predictions.csv"
)


# ============================================================
# REQUEST SETTINGS
# ============================================================

# Chỉ gửi request tiếp theo sau khi
# đã nhận được response hoàn chỉnh.

DELAY_AFTER_RESPONSE = 4.5

# Retry tối đa với các lỗi tạm thời
MAX_RETRY = 1000

# Thời gian chờ tối đa giữa các lần retry
MAX_WAIT_SECONDS = 60


# ============================================================
# CREATE GROQ CLIENT
# ============================================================

client = AsyncGroq(
    api_key=GROQ_API_KEY
)


print("=" * 60)
print("Groq Benchmark")
print("=" * 60)
print("Model      :", MODEL_NAME)
print("Checkpoint :", CHECKPOINT_CSV)
print("Output     :", OUTPUT_CSV)
print("=" * 60)
```

```
/opt/anaconda3/envs/LLM/lib/python3.11/site-packages/tqdm/auto.py:21:
TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
============================================================
Groq Benchmark
============================================================
Model      : qwen/qwen3.6-27b
Checkpoint : ./Training_Data/groq_qwen36_checkpoint.csv
Output     : ./Training_Data/Baseline_Groq_Qwen3.6-27B_predictions.csv
============================================================
```

** Canvas 2 **

In [2]:

```
# ============================================================
# LOAD DATASET
# ============================================================

dataset = []

print("\nLoading dataset...")
```

```
with open(TEST_FILE, "r", encoding="utf-8") as f:

    for line in f:

        line = line.strip()

        if line:

            dataset.append(
                json.loads(line)
            )


TOTAL_SAMPLES = len(dataset)

print(
    f"Loaded {TOTAL_SAMPLES} samples."
)


# ====================================================
# LOAD CHECKPOINT
# ====================================================

def load_checkpoint():

    predictions = []

    references = []

    start_idx = 0


    if not os.path.exists(CHECKPOINT_CSV):

        print(
            "\nNo checkpoint found."
        )

        print(
            "Start from sample 1."
        )

        return (
            predictions,
            references,
            start_idx
        )


    print(
        "\nCheckpoint detected."
    )


    try:

        checkpoint = pd.read_csv(
            CHECKPOINT_CSV,
            encoding="utf-8"
```

```
        )


        if "prediction" not in checkpoint.columns:

            raise RuntimeError(
                "Prediction column not found."
            )


        if "ground_truth" not in checkpoint.columns:

            raise RuntimeError(
                "Ground truth column not found."
            )


        checkpoint["prediction"] = (
            checkpoint["prediction"]
            .fillna("")
        )


        checkpoint["ground_truth"] = (
            checkpoint["ground_truth"]
            .fillna("")
        )


        predictions = (
            checkpoint["prediction"]
            .tolist()
        )


        references = (
            checkpoint["ground_truth"]
            .tolist()
        )


        start_idx = len(
            predictions
        )


        if start_idx > TOTAL_SAMPLES:

            raise RuntimeError(
                "Checkpoint larger than dataset."
            )


        print(
            f"Recovered "
            f"{start_idx} completed samples."
        )


        if start_idx < TOTAL_SAMPLES:
```

```
            print(
                f"Resume from sample "
                f"{start_idx + 1}/{TOTAL_SAMPLES}"
            )

        else:

            print(
                "All samples already completed."
            )


        return (
            predictions,
            references,
            start_idx
        )


    except Exception as e:

        print(
            "\nFailed to read checkpoint."
        )

        print(e)

        print(
            "\nRestart from beginning."
        )

        return (
            [],
            [],
            0
        )


# ====================================================
# SAVE CHECKPOINT
# ====================================================

def save_checkpoint(
    predictions,
    references
):

    completed = len(
        predictions
    )


    checkpoint = pd.DataFrame(
        {

            "instruction":
            [
                x["instruction"]
                for x in dataset[:completed]
            ],
```

```
            "input":
            [
                x["input"]
                for x in dataset[:completed]
            ],

            "ground_truth":
            references,

            "prediction":
            predictions

        }
    )


    checkpoint.to_csv(
        CHECKPOINT_CSV,
        index=False,
        encoding="utf-8"
    )


    print(
        f"💾 Checkpoint saved "
        f"({completed}/{TOTAL_SAMPLES})"
    )
```

```
Loading dataset...
Loaded 121 samples.
```

** Canvas 3 **

In [3]:

```
# ======================================================
# SEND REQUEST TO GROQ
# NEVER SKIP A SAMPLE
# ======================================================

async def send_and_wait_for_response(
        sample,
        idx,
        total
):

    prompt = f"""
Instruction:
{sample['instruction']}

Input:
{sample['input']}

Answer:
"""

    attempt = 1

    while True:

        try:

            print(
                f"\n[{idx+1}/{total}] "
```

```
            f"Sending request to Groq "
            f"(attempt {attempt})"
        )


        completion = await client.chat.completions.create(

            model=MODEL_NAME,

            messages=[
                {
                    "role": "user",
                    "content": prompt
                }
            ],

            temperature=0.0,

            max_completion_tokens=2048,

            top_p=1.0,

            stream=False

        )


        if (
            completion is None
            or len(completion.choices) == 0
        ):
            raise RuntimeError(
                "Empty response from Groq."
            )


        prediction = (
            completion
            .choices[0]
            .message
            .content
        )


        if prediction is None:

            raise RuntimeError(
                "Groq returned None."
            )


        prediction = prediction.strip()


        if prediction == "":

            raise RuntimeError(
                "Groq returned empty text."
            )
```

```
            print("✅ Response received")

            preview = prediction.replace(
                "\n",
                " "
            )[:120]

            print(preview + "...")

            return prediction


        except Exception as e:

            error_message = str(e)

            print(
                "\n❌ Groq Error:"
            )

            print(error_message)


            # ========================================
            # QUOTA EXCEEDED -> STOP PROGRAM
            # ========================================

            if (
                "429" in error_message
                or "rate_limit" in error_message.lower()
                or "quota" in error_message.lower()
                or "resource_exhausted" in error_message.lower()
            ):

                print("\n" + "=" * 60)
                print("❌ QUOTA EXCEEDED")
                print("Program stopped.")
                print("Checkpoint is preserved.")
                print("Run again after quota resets.")
                print("=" * 60)

                raise SystemExit(
                    "Stopped because quota was exceeded."
                )


            # ========================================
            # OTHER ERRORS -> RETRY
            # ========================================

            wait_time = min(
                MAX_WAIT_SECONDS,
                attempt * 5
            )

            print(
                f"Retry after {wait_time} seconds..."
            )
```

```
            await asyncio.sleep(
                wait_time
            )

            attempt += 1
```

In [4]:

```
# ======================================================
# EVALUATION
# ======================================================

async def evaluate_and_save(
        predictions,
        references
):

    print(
        "\n========================================"
    )

    print(
        "Computing evaluation metrics..."
    )


    assert len(predictions) == len(references)

    assert len(predictions) == TOTAL_SAMPLES


    # ================================================
    # BERTScore
    # ================================================

    print("\nComputing BERTScore...")

    P, R, F1 = score(

        predictions,

        references,

        lang="en",

        verbose=True

    )


    # ================================================
    # ROUGE-L
    # ================================================

    print("\nComputing ROUGE-L...")


    scorer = rouge_scorer.RougeScorer(

        ["rougeL"],

        use_stemmer=True

    )
```

```
rouge_precision = []
rouge_recall = []
rouge_f1 = []

for pred, ref in zip(
    predictions,
    references
):

    result = scorer.score(
        ref,
        pred
    )

    rouge = result["rougeL"]

    rouge_precision.append(
        rouge.precision
    )

    rouge_recall.append(
        rouge.recall
    )

    rouge_f1.append(
        rouge.fmeasure
    )

# ================================================
# SAVE RESULT
# ================================================

print("\nSaving CSV...")

final_df = pd.DataFrame(
    {
```

```
            "instruction":
            [
                x["instruction"]
                for x in dataset
            ],
            "input":
            [
                x["input"]
                for x in dataset
            ],
            "ground_truth":
            references,
            "prediction":
            predictions,
            "bert_precision":
            P.cpu().numpy(),
            "bert_recall":
            R.cpu().numpy(),
            "bert_f1":
            F1.cpu().numpy(),
            "rougeL_precision":
            rouge_precision,
            "rougeL_recall":
            rouge_recall,
            "rougeL_f1":
            rouge_f1
```

```
        }
    )

    final_df.to_csv(
        OUTPUT_CSV,
        index=False,
        encoding="utf-8"
    )

    # ================================================
    # REPORT
    # ================================================
    print("\n========================================")
    print("Average BERTScore")
    print("----------------------------------------")
    print(
        f"Precision : {P.mean().item():.4f}"
    )
    print(
        f"Recall    : {R.mean().item():.4f}"
    )
    print(
        f"F1        : {F1.mean().item():.4f}"
    )

    print("\n========================================")
    print("Average ROUGE-L")
    print("----------------------------------------")
    print(
        f"Precision : "
        f"{sum(rouge_precision)/len(rouge_precision):.4f}"
    )
```

```
    print(
        f"Recall    : "
        f"{sum(rouge_recall)/len(rouge_recall):.4f}"
    )

    print(
        f"F1        : "
        f"{sum(rouge_f1)/len(rouge_f1):.4f}"
    )

    print("\n========================================")
    print("Evaluation completed.")
    print(f"Output CSV : {OUTPUT_CSV}")
    print("========================================")
```

** Canvas 4 **

In [5]:

```
# ======================================================
# MAIN LOOP
# ======================================================

async def main():

    # ----------------------------------------------
    # LOAD CHECKPOINT
    # ----------------------------------------------

    (
        predictions,
        references,
        start_idx
    ) = load_checkpoint()


    # ----------------------------------------------
    # ALL SAMPLES COMPLETED
    # ----------------------------------------------

    if start_idx >= TOTAL_SAMPLES:

        print("\nAll samples already completed.")
        print("Start evaluation...\n")

        await evaluate_and_save(
            predictions,
            references
        )

        return
```

```
print("\n========================================")
print("Starting inference...")
print("========================================")


# ----------------------------------------
# INFERENCE LOOP
# ----------------------------------------

for idx in range(
    start_idx,
    TOTAL_SAMPLES
):

    sample = dataset[idx]


    # ----------------------------------------
    # WAIT UNTIL RESPONSE IS RECEIVED
    # ----------------------------------------

    prediction = await send_and_wait_for_response(

        sample=sample,

        idx=idx,

        total=TOTAL_SAMPLES

    )


    # ----------------------------------------
    # SAVE RESULT IN MEMORY
    # ----------------------------------------

    predictions.append(
        prediction
    )

    references.append(
        sample["output"]
    )


    print(
        f"Completed "
        f"{len(predictions)}/{TOTAL_SAMPLES}"
    )


    # ----------------------------------------
    # SAVE CHECKPOINT
    # ----------------------------------------

    if (
        len(predictions) % 5 == 0
        or idx == TOTAL_SAMPLES - 1
    ):

        save_checkpoint(
```

```
                predictions,
                references
            )


        # ----------------------------------------
        # WAIT BEFORE NEXT REQUEST
        # ----------------------------------------

        if idx < TOTAL_SAMPLES - 1:

            print(
                f"Waiting "
                f"{DELAY_AFTER_RESPONSE}s..."
            )

            await asyncio.sleep(
                DELAY_AFTER_RESPONSE
            )


    # ----------------------------------------
    # ALL RESPONSES RECEIVED
    # ----------------------------------------

    print("\n========================================")
    print("All Groq responses completed.")
    print("Start evaluation...")
    print("========================================\n")


    await evaluate_and_save(

        predictions,

        references

    )
```

** Canvas 6 **

In [6]:

```
try:

    await main()

except KeyboardInterrupt:

    print("\nProgram interrupted by user.")
    print("Checkpoint has been preserved.")

except SystemExit as e:

    print(e)

except Exception as e:

    print(type(e).__name__)
    print(e)
    raise
```

```
Checkpoint detected.
Recovered 115 completed samples.
Resume from sample 116/121
```

```
========================================
Starting inference...
========================================

[116/121] Sending request to Groq (attempt 1)
✅ Response received
<think> Here's a thinking process:  1.  **Understand User Goal:** The user
wants to compare two itemsets, A `[41, 11610]...
Completed 116/121
Waiting 4.5s...

[117/121] Sending request to Groq (attempt 1)
✅ Response received
<think> Here's a thinking process:  1.  **Analyze User Input:**    -
**Question:** How does itemset [9435] perform compa...
Completed 117/121
Waiting 4.5s...

[118/121] Sending request to Groq (attempt 1)
✅ Response received
<think> Here's a thinking process:  1.  **Understand User Goal:** The user
wants to compare two itemsets, A `[41, 994]` ...
Completed 118/121
Waiting 4.5s...

[119/121] Sending request to Groq (attempt 1)
✅ Response received
<think> Here's a thinking process:  1.  **Understand the User's Goal:** The
user wants to determine if the itemset `[39,...
Completed 119/121
Waiting 4.5s...

[120/121] Sending request to Groq (attempt 1)
✅ Response received
<think> Here's a thinking process:  1.  **Understand the User's Request:**    -
**Input:** An itemset `[39, 48, 5439]` w...
Completed 120/121
💾 Checkpoint saved (120/121)
Waiting 4.5s...

[121/121] Sending request to Groq (attempt 1)
✅ Response received
<think> Here's a thinking process:  1.  **Understand User Goal:** The user
wants an analysis of why the itemset `[48, 41...
Completed 121/121
💾 Checkpoint saved (121/121)

========================================
All Groq responses completed.
Start evaluation...
========================================


========================================
Computing evaluation metrics...
```

```
Computing BERTScore...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a
HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|██████████| 389/389 [00:00<00:00, 6981.83it/s]
[transformers] RobertaModel LOAD REPORT from: roberta-large
Key                         | Status     |
----------------------------+------------+-
lm_head.layer_norm.bias     | UNEXPECTED |
lm_head.dense.bias          | UNEXPECTED |
lm_head.dense.weight        | UNEXPECTED |
lm_head.layer_norm.weight   | UNEXPECTED |
lm_head.bias                | UNEXPECTED |
pooler.dense.bias           | MISSING    |
pooler.dense.weight         | MISSING    |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture;
not ok if you expect identical arch.
- MISSING:      those params were newly initialized because missing from the
checkpoint. Consider training on your downstream task.
calculating scores...
computing bert embedding.
100%|██████████| 4/4 [00:19<00:00,  4.79s/it]
computing greedy matching.
100%|██████████| 2/2 [00:00<00:00, 25.77it/s]
done in 19.25 seconds, 6.29 sentences/sec

Computing ROUGE-L...

Saving CSV...

========================================
Average BERTScore
----------------------------------------
Precision : 0.7862
Recall    : 0.8485
F1        : 0.8160

========================================
Average ROUGE-L
----------------------------------------
Precision : 0.0529
Recall    : 0.5473
F1        : 0.0946

========================================
Evaluation completed.
Output CSV : ./Training_Data/Baseline_Groq_Qwen3.6-27B_predictions.csv
========================================
```

** Đọc kết quả file CSV **

In [24]:

```python
import pandas as pd
import re

# ============================================================
# Extract numbers
# ============================================================

def extract_numbers(text):
    """
```

```
    Extract all integers and decimal numbers from text.
    """
    return re.findall(r"\d+(?:\.\d+)?", text)


# ============================================================
# Numerical Faithfulness
# ============================================================

def numerical_faithfulness(input_text, prediction_text):

    input_numbers = extract_numbers(input_text)
    prediction_numbers = extract_numbers(prediction_text)

    # Prediction không sinh số
    if len(prediction_numbers) == 0:
        return {
            "InputNumbers": input_numbers,
            "PredictionNumbers": prediction_numbers,
            "HallucinatedNumbers": [],
            "CorrectNumbers": 0,
            "PredictionCount": 0,
            "NumericalFaithfulness": 1.0
        }

    correct = 0
    hallucinated = []

    for num in prediction_numbers:

        if num in input_numbers:
            correct += 1
        else:
            hallucinated.append(num)

    score = correct / len(prediction_numbers)

    return {
        "InputNumbers": input_numbers,
        "PredictionNumbers": prediction_numbers,
        "HallucinatedNumbers": hallucinated,
        "CorrectNumbers": correct,
        "PredictionCount": len(prediction_numbers),
        "NumericalFaithfulness": score
    }


# ============================================================
# Evaluation
# ============================================================


df = pd.read_csv(OUTPUT_CSV, encoding="utf-8")

results = []

for sample, prediction in df[["input",
"prediction"]].itertuples(index=False):

    result = numerical_faithfulness(
        sample,
```

```
        prediction
    )

    results.append(result)

result_df = pd.DataFrame(results)

print("=" * 60)
print("Average Numerical Faithfulness:",
      result_df["NumericalFaithfulness"].mean())
print("=" * 60)

result_df.to_csv(
    "NumericalFaithfulness_Qwen-3.6-27b.csv",
    index=False,
    encoding="utf-8-sig"
)
```

```
============================================================
Average Numerical Faithfulness: 0.5555798078481599
============================================================
```

** Numerical Faithfulness **

** Explanation Completeness **

In [26]:

```
import re

# ============================================================
# Keywords
# ============================================================

SUPPOR_KEYWORDS = [
    "support",
    "supports",
    "support threshold",
    "support exceeds",
    "purchased frequently",
    "frequently occurring",
    "frequent itemset"
]

AVG_LENGTH_KEYWORDS = [
    "average support-transaction length",
    "average transaction length",
    "longer transaction",
    "shorter transaction",
    "longer transactions",
    "shorter transactions",
    "average length"
]

FI_SCORE_KEYWORDS = [

    "fi score",

    "score",

    "overall performance",

    "better performance",

    "higher score",
```

```
    "lower score",
    "overall score",
    "business impact",
    "overall impact"
]


# ============================================================
# Utility
# ============================================================

def contains_keywords(text, keywords):
    text = text.lower()
    return any(keyword.lower() in text for keyword in keywords)


# ============================================================
# Support
# ============================================================

def evaluate_support(prediction):
    return int(
        contains_keywords(
            prediction,
            SUPPOR_KEYWORDS
        )
    )


# ============================================================
# Average Support Length
# ============================================================

def evaluate_average_length(prediction):
    return int(
        contains_keywords(
            prediction,
            AVG_LENGTH_KEYWORDS
        )
    )


# ============================================================
# FI Score
# ============================================================

def evaluate_fi_score(prediction):
    return int(
        contains_keywords(
            prediction,
            FI_SCORE_KEYWORDS
```

```
        )
    )


# ============================================================
# Overall
# ============================================================

def explanation_completeness(prediction):

    support = evaluate_support(prediction)

    avg = evaluate_average_length(prediction)

    fi = evaluate_fi_score(prediction)

    return {
        "support": support,
        "average_length": avg,
        "fi_score": fi,
        "ExplanationCompleteness":
            (support + avg + fi) / 3
    }


rows = []

for prediction in df["prediction"]:

    rows.append(
        explanation_completeness(
            prediction
        )
    )

result_df = pd.DataFrame(rows)

print("=" * 50)
print(result_df.mean(numeric_only=True))

result_df.to_csv(
    "./Training_Data/ExplanationCompleteness_Qwen-3.6-27b.csv",
    index=False
)
```

```
==================================================
support                     1.000000
average_length              0.991736
fi_score                    0.818182
ExplanationCompleteness     0.936639
dtype: float64
```

** Numerical Correctness **

In [30]:

```
import re
import pandas as pd

# ============================================================
# CONFIG
# ============================================================

TOLERANCE = 0.1
```

```python
# ============================================================
# Remove Itemset IDs
# ============================================================

def remove_itemsets(text):
    """
    Remove itemset IDs.

    Example:
        [39, 41, 1799]
        [6009]
    """

    if text is None:
        return ""

    return re.sub(r"\[[^\]]*\]", "", text)


# ============================================================
# Extract Numbers
# ============================================================

def extract_numbers(text):
    """
    Extract all numerical values.
    """

    text = remove_itemsets(text)

    pattern = r"-?\d+(?:\.\d+)?"

    numbers = re.findall(pattern, text)

    return [float(x) for x in numbers]


# ============================================================
# Direct Match
# ============================================================

def is_direct_match(pred, input_numbers):

    for x in input_numbers:

        if abs(pred - x) <= TOLERANCE:
            return True

    return False


# ============================================================
# Division Match
# ============================================================

def is_division_match(pred, input_numbers):

    n = len(input_numbers)

    for i in range(n):
```

```
        for j in range(n):

            if i == j:
                continue

            a = input_numbers[i]
            b = input_numbers[j]

            # if abs(b) <= TOLERANCE:
            #     continue

            ratio = a / b

            if abs(pred - ratio) <= TOLERANCE:
                return True

    return False


# ============================================================
# Numerical Correctness
# ============================================================

def numerical_correctness(input_text, prediction_text):

    input_numbers = extract_numbers(input_text)

    pred_numbers = extract_numbers(prediction_text)

    direct_match = 0
    division_match = 0
    incorrect = 0

    for pred in pred_numbers:

        # 1. Có trực tiếp trong input
        if is_direct_match(pred, input_numbers):

            direct_match += 1

        # 2. Không có nhưng bằng phép chia
        elif is_division_match(pred, input_numbers):

            division_match += 1

        # 3. Sai hoàn toàn
        else:

            incorrect += 1

    correct = direct_match + division_match

    if len(pred_numbers) == 0:
        score = 0
    else:
        score = correct / len(pred_numbers)

    return {

        "input_numbers": len(input_numbers),
```

```
        "predicted_numbers": len(pred_numbers),

        "direct_match": direct_match,

        "division_match": division_match,

        "correct": correct,

        "incorrect": incorrect,

        "NumericalCorrectness": score
    }


# ============================================================
# Evaluate
# ============================================================

rows = []

for input_text, prediction in df[["input",
"prediction"]].itertuples(index=False):

    rows.append(
        numerical_correctness(
            input_text,
            prediction
        )
    )

df = pd.DataFrame(rows)

print(df)

print("="*70)
print("Average")
print("="*70)
print(df.mean(numeric_only=True))

print()

print("="*70)
print("Percentage")
print("="*70)

print(f"Average numbers in Input      : {df['input_numbers'].mean():.2f}")
print(f"Average numbers in Prediction :
{df['predicted_numbers'].mean():.2f}")
print(f"Direct Match                  : {df['direct_match'].mean():.2f}")
print(f"Division Match                : {df['division_match'].mean():.2f}")
print(f"Correct Numbers               : {df['correct'].mean():.2f}")
print(f"Incorrect Numbers             : {df['incorrect'].mean():.2f}")
print(f"Numerical Correctness         :
{df['NumericalCorrectness'].mean()*100:.2f}%")

df.to_csv(
    f"./Training_Data/NumericalCorrectness_Qwen-3.6-27b.csv",
    index=False
)

print("\nSaved to NumericalCorrectness.csv")
```

```
     input_numbers  predicted_numbers  direct_match  division_match  correct  \
0               16                 76            50               8       58
1                8                 47            40               0       40
2               16                 53            44               7       51
3               16                 66            58               2       60
4                8                 65            58               2       60
..             ...                ...           ...             ...      ...
116             17                 62            54               7       61
117             16                 62            52               8       60
118              8                 42            35               2       37
119              8                 58            51               1       52
120              8                 76            60               2       62

     incorrect  NumericalCorrectness
0           18              0.763158
1            7              0.851064
2            2              0.962264
3            6              0.909091
4            5              0.923077
..         ...                   ...
116          1              0.983871
117          2              0.967742
118          5              0.880952
119          6              0.896552
120         14              0.815789

[121 rows x 7 columns]
========================================================================
Average
========================================================================
input_numbers           12.181818
predicted_numbers       64.917355
direct_match            54.454545
division_match           4.644628
correct                 59.099174
incorrect                5.818182
NumericalCorrectness     0.913012
dtype: float64

========================================================================
Percentage
========================================================================
Average numbers in Input      : 12.18
Average numbers in Prediction : 64.92
Direct Match                  : 54.45
Division Match                : 4.64
Correct Numbers               : 59.10
Incorrect Numbers             : 5.82
Numerical Correctness         : 91.30%

Saved to NumericalCorrectness.csv
```

In [ ]: